

Portfolio Optimization using Reinforcement learning and Technical Analysis

Lakshmi Narayanan S K
UG Thesis Report

Under the supervision of
Dr.Charu Sharma
Department of Mathematics Shiv Nadar Institute of Eminence

1 Abstract

This study presents a reinforcement learning-based model for portfolio optimization, targeting stocks from two sectors of the National Stock Exchange (NSE). The model is driven by a profit-and-loss-oriented reward function and learns trading policies based on stock behavior patterns. To benchmark the performance of the reinforcement learning approach, a traditional technical analysis-based strategy—incorporating indicators such as moving averages, Bollinger Bands, and Moving Average Convergence Divergence (MACD)—is used for comparison. The research further evaluates the effectiveness of various deep reinforcement learning algorithms in trading decisions, offering insights into their comparative performance against indicator-driven strategies.

2 Data Collection

The stock data for this research is collected from Yahoo finance. Instead of downloading the data from the website as an excel file we use the 'yfinance' library in python to directly access the data. The data primarily contains historical data of Open, High, Low, Close prices for companies along with the dates in which prices were recorded. We have considered to use the price data for the past 5 years of 6 companies from the pharmaceutical sector.

S. No	Company Name
1	CIPLA LTD.
2	AURO PHARMA
3	LAURUS LABS
4	SUN PHARMA
5	ZYDUS LIFE
6	ABBOTT LTD.

Table 1: Selected Company Names.

3 Technical analysis indicators

Technical analysis is a method of evaluating statistical trends in trading activity, typically involving price movement and volume. It is used to identify trading and investment opportunities. In this study, we compare the commonly used indicators to reinforcement learning algorithms.

3.1 Moving averages

A moving average is a value that represents a set of past values. Moving average has several types of which we will use only Simple moving average(SMA) and Exponential moving average(EMA). Technical analysts use moving averages to smooth out shorter fluctuations and focus on the trend that fits with the investor's time horizon.

3.1.1 Simple Moving average

To calculate the 30-day simple moving average (SMA) of a stock, we compute the mean of the closing prices over the past 30 days. For example, to calculate the SMA-30 for day 31, we take the average of the closing prices from day 2 to day 31.

$$SMA_{30} = \sum_{i=1}^{30} \frac{Closingprice_i}{30} \quad (1)$$

In this research, we have used 30-day, 60-day and 200-day moving averages to assess stock trends. In a typical year, there are 52 weeks, each with 5 working days, resulting in 260 trading days, excluding holidays. Therefore, the 200-day moving average is particularly effective as a long-term indicator, reflecting broader market trends.

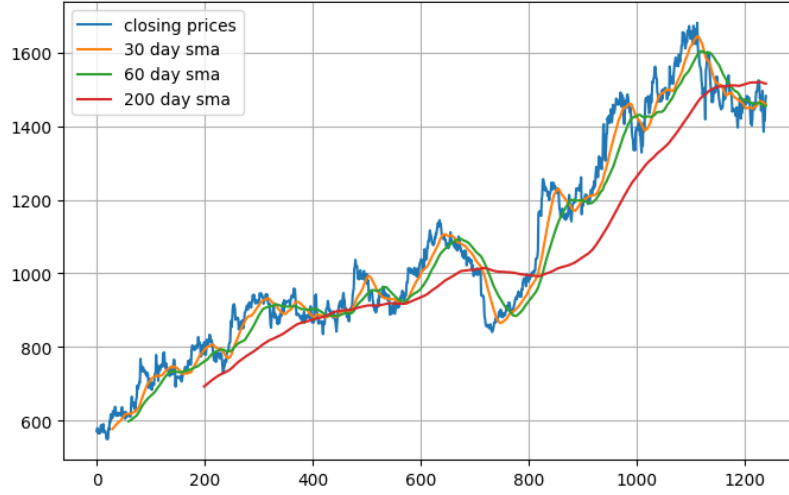


Figure 1: Moving averages for CIPLA 9/4/2020 to 9/4/2025

3.1.2 Exponential Moving average

If the most recent price reflects little change but the earlier prices show considerable change, then the moving average can be made more effective by decreasing the significance of older data.

$$Weight_{current} = \frac{2}{numberofdaysinM.A. + 1} \quad (2)$$

$$Weight_{ma} = 1 - Weight_{current} \quad (3)$$

$$EMA_{day_i} = (Weight_{current} \times Closingprice_{day_i}) + (Weight_{ma} \times EMA_{day_i-1}) \quad (4)$$

3.2 Bollinger bands

Bands are envelopes around a moving average which are calculated to adjust for the price volatility around the moving average. They shrink when prices are calm and expand when prices are volatile. Bollinger bands are calculated using the standard deviation around a simple moving average. For example, let us calculate Bollinger bands with the 30 day moving average,

$$UpperBollingerband = SMA_{30} + (2 \times \sigma_{SMA_{30}}) \quad (5)$$

$$\text{LowerBollingerband} = SMA_{30} - (2 \times \sigma_{SMA_{30}}) \quad (6)$$

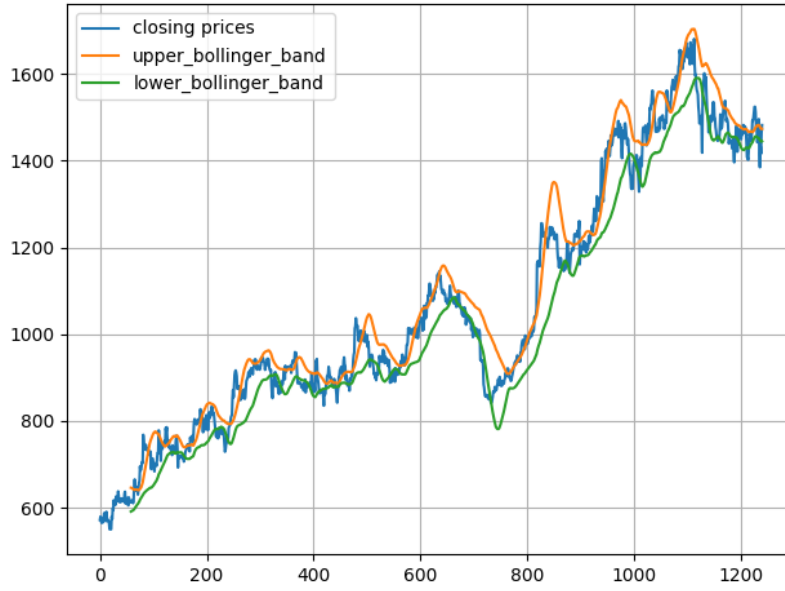


Figure 2: Bollinger bands for CIPLA 9/4/2020 to 9/4/2025

3.3 Moving average Convergence Divergence(MACD)

MACD is calculated by subtracting the Exponential Moving Average (EMA) of the 26-period from the 12-period EMA. The resulting value is then plotted along with a 9-period EMA of the MACD itself, known as the signal line.

$$M.A.C.D. = EMA_{12} - EMA_{26} \quad (7)$$

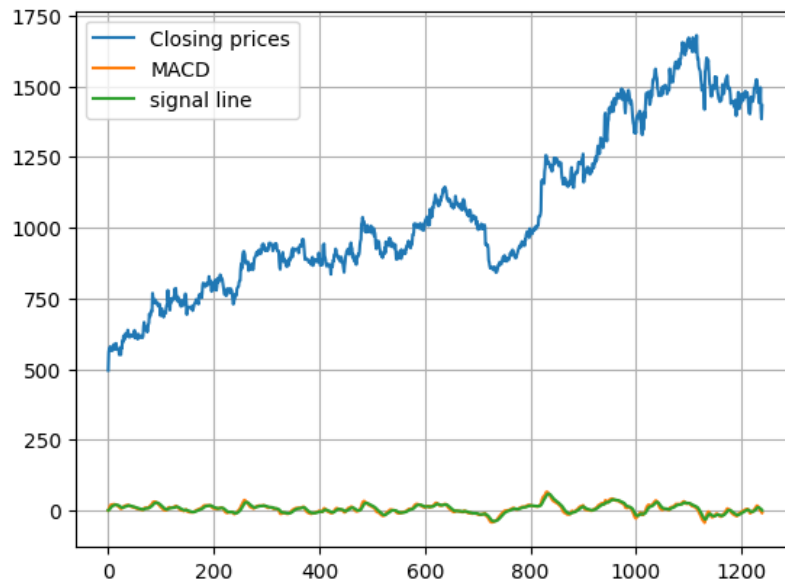


Figure 3: M.A.C.D. for CIPLA 9/4/2020 to 9/4/2025

When the MACD line is above the signal line, it typically indicates an upward trend, while when it is below the signal line, it suggests a downward trend.

4 Reinforcement Learning

Reinforcement learning is a subfield of machine learning where an agent learns by interacting with its environment. The agent begins in an initial state and performs an action to transition to a new state. In each state, the environment provides a reward based on the action taken. The objective of a reinforcement learning algorithm is to maximize the cumulative reward as the agent progresses toward the terminal (final) state.

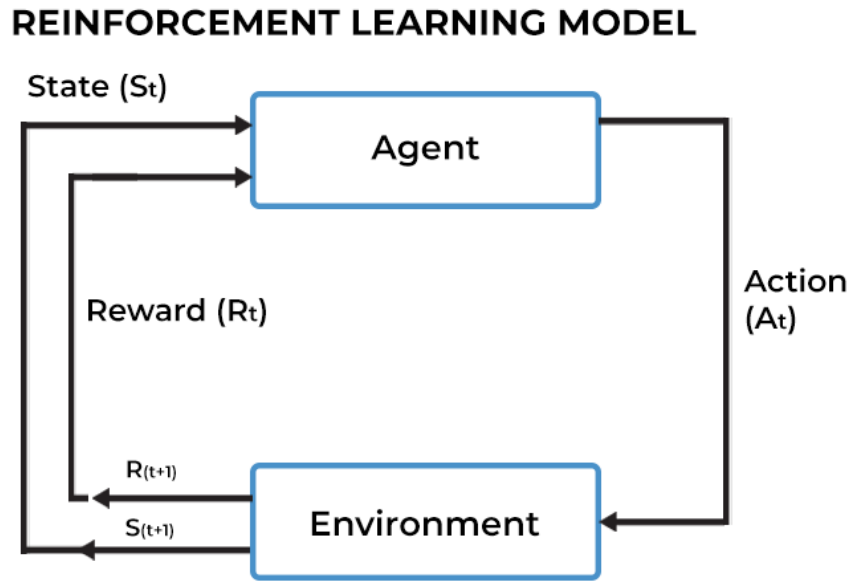


Figure 4:

s_t = state at time t
 a_t = action performed in state s_t
 r_t = reward at state s_t
 s_{t+1} = state at time $t + 1$

If the machine goes through a sequence of states, then the return is given by (let $r_1, r_2, r_3, \dots$ be the respective rewards in states $s_1, s_2, s_3, \dots$).

$$Return = r_1 + \Gamma r_2 + \Gamma^2 r_3 + \dots \quad (8)$$

The Γ used here represents the discount factor (which is a percentage) that is used to discount rewards in the future. This is because with every additional passing timestep you get only less credit for the reward than in the previous step.

4.1 Policies for trading strategy

In Reinforcement learning a policy(π) is a function that specifies the action an agent should take in each state. A policy defines the agent's strategy for choosing actions. Policies are of 2 types

- **Deterministic** policy maps a state to one possible action.
- **Stochastic** policy is a probability distribution over actions given a state. The policy outputs the probabilities of choosing each possible action, and the action is selected randomly according to this distribution.

For this research we have chosen four different policies (all deterministic strategies), each following a trading strategy based on the indicator chosen.

4.1.1 Moving averages trading strategy

- When the shorter term M.A. crosses above the longer term M.A. then it is a **buy signal**.
- When the shorter term M.A. crosses below the longer term M.A. then it is a **sell signal**.
- When there is no such crossover then the stock should be on hold.

4.1.2 Bollinger bands trading strategy

- When the closing price exceeds the upper Bollinger band then it is a **sell signal**.
- When the closing price precedes the lower Bollinger band then it is a **buy signal**.
- When the closing price is between the 2 Bollinger bands then the stock should be on hold.

4.1.3 M.A.C.D trading strategy

- When the M.A.C.D. line crosses above the signal line then it is a **buy signal**.
- When the M.A.C.D. line crosses below the signal line then it is a **sell signal**.
- When there are no crossovers then the stock should be on hold.

4.1.4 Greedy Policy

According to the greedy algorithm, the action which gives the most return in the next consecutive states should be chosen for each timestamp till the terminal state.

4.2 Framework in Python for deterministic policies

With the above given strategies each of the policy maps a given state to an action from the possible actions. The state (for a given day) is formatted as a matrix containing number of stocks in each company along with the technical analysis metrics for each company in that given day. The state matrix in a given timestamp is as given below,

$$\begin{pmatrix} n_1 & n_2 & n_3 & n_4 & n_5 & n_6 & n_7 \\ c_1 & c_2 & c_3 & c_4 & c_5 & c_6 & c_7 \\ MA30_1 & MA30_2 & MA30_3 & MA30_4 & MA30_5 & MA30_6 & MA30_7 \\ MA60_1 & MA60_2 & MA60_3 & MA60_4 & MA60_5 & MA60_6 & MA60_7 \\ MA200_1 & MA200_2 & MA200_3 & MA200_4 & MA200_5 & MA200_6 & MA200_7 \\ UBB_1 & UBB_2 & UBB_3 & UBB_4 & UBB_5 & UBB_6 & UBB_7 \\ LBB_1 & LBB_2 & LBB_3 & LBB_4 & LBB_5 & LBB_6 & LBB_7 \\ MACD_1 & MACD_2 & MACD_3 & MACD_4 & MACD_5 & MACD_6 & MACD_7 \\ SGN_1 & SGN_2 & SGN_3 & SGN_4 & SGN_5 & SGN_6 & SGN_7 \end{pmatrix}$$

n_i = Number of stocks in the ith company
 c_i = Closing price of ith company
 $MA30_i$ = 30 day moving average of ith company
 UBB_i = Upper Bollinger band value of ith company
 $MACD_i$ = M.A.C.D. value of ith company
 SGN_i = MACD signal value of ith company

In addition to the estimates in the matrix, another crucial metric in the state is the **liquidity(cash)** held which is given by a list [cash, state] in Python. The reward function for each state is defined as the percent gain or loss between the previous and current state. Consequently, all states, except the initial one, are associated with a reward.

4.3 Reinforcement learning by using policy gradient method

For the implementation of reinforcement learning, there are two important functions **state value function** ($v_\pi(s)$) and **state action value function** ($q_\pi(s, a)$). We use the closing prices of the historical stock data of six companies in Table 1 of the medical sector. For this, the state(S_t) needs to be converted to a tensor that looks the following.

$$\begin{bmatrix} \text{cash} & n_1 & c_1 \\ 0 & n_2 & c_2 \\ 0 & n_3 & c_3 \\ 0 & n_4 & c_4 \\ 0 & n_5 & c_5 \\ 0 & n_6 & c_6 \end{bmatrix}$$

Where $n_1, n_2, n_3, \dots$ are the number of stock holdings of the respective companies at that time t and each action is taken to be a vector (integer matrix) as $A_t = [\mathbf{a1}, \mathbf{a2}, \mathbf{a3}, \mathbf{a4}, \mathbf{a5}, \mathbf{a6}]$ where each integer in this vector represents a trading action for that particular stock. Positive integer for buy, negative for sell, and zero for hold.

$$\pi(a | s) = \text{Prob}(A_t = a \text{ if } S_t = s) \quad (9)$$

$$G_t = R_{t+1} + \gamma * R_{t+2} + \gamma * R_{t+3} \quad (10)$$

$$v_\pi(s) = E[G_t | S_t = s] v_\pi(s) = E\left[\sum_{k=1}^N \gamma^k * R_{t+k+1} | S_t = s\right] \quad (11)$$

$$q_\pi(s, a) = E[G_t | S_t = s, A_t = a] \quad (12)$$

4.3.1 Monte Carlo Policy Gradient theorem

For defining the values for probability for the actions given state we use the softmax function which is used for classification problems involving more than 2 classes.

$$\pi(a | s, \theta) = \frac{e^{h(s, a, \theta)}}{\sum_b e^{h(s, b, \theta)}} \quad (13)$$

$$h(s, a, \theta) = \theta^\top x(s, a) \quad (14)$$

The cost function is taken to be $J(\theta) = v_\pi(s_0)$. We calculate the gradients with respect to θ because we use it as the learning weights in the context of deep reinforcement learning.

$$\begin{aligned}
\nabla v_\pi(s) &= \nabla \left(\sum_a \pi(a | s) q_\pi(s, a) \right) \\
&= \sum_a [\nabla \pi(a | s) \cdot q_\pi(s, a) + \pi(a | s) \cdot \nabla q_\pi(s, a)] \\
&= \sum_a \left[\nabla \pi(a | s) \cdot q_\pi(s, a) + \pi(a | s) \cdot \nabla \sum_{s', r} p(s', r | s, a) (r + v_\pi(s')) \right] \\
&= \sum_a \left[\nabla \pi(a | s) \cdot q_\pi(s, a) + \pi(a | s) \cdot \sum_{s'} p(s' | s, a) \nabla v_\pi(s') \right] \\
&= \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \Pr(s \rightarrow x, k, \pi) \sum_a \nabla \pi(a | x) q_\pi(x, a)
\end{aligned} \tag{15}$$

$$\begin{aligned}
\nabla J(\theta) &= \nabla v_\pi(s_0) \\
&= \sum_s \sum_{k=0}^{\infty} \Pr(s_0 \rightarrow s, k, \pi) \sum_a \nabla \pi(a | s) q_\pi(s, a) \\
&= \sum_s \eta(s) \sum_a \nabla \pi(a | s) q_\pi(s, a) \\
&= \sum_{s_0} \eta(s_0) \sum_s \frac{\eta(s)}{\eta(s_0)} \sum_a \nabla \pi(a | s) q_\pi(s, a) \\
&= \sum_{s_0} \eta(s_0) \sum_s \mu(s) \sum_a \nabla \pi(a | s) q_\pi(s, a) \\
&\propto \sum_s \mu(s) \sum_a \nabla \pi(a | s) q_\pi(s, a)
\end{aligned} \tag{16}$$

The policy gradient theorem gives an exact expression proportional to the gradient. All that is needed is some way of sampling whose expectation equals or approximates this expression.

$$\nabla_\theta J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_\theta \pi(a | s, \theta) = \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla_\theta \pi(a | S_t, \theta) \right] \tag{17}$$

$$\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_\pi \left[\sum_a \pi(a | S_t, \theta) q_\pi(S_t, a) \frac{\nabla_\theta \pi(a | S_t, \theta)}{\pi(a | S_t, \theta)} \right] \\
&= \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \frac{\nabla_\theta \pi(A_t | S_t, \theta)}{\pi(A_t | S_t, \theta)} \right] \\
&= \mathbb{E}_\pi \left[G_t \frac{\nabla_\theta \pi(A_t | S_t, \theta)}{\pi(A_t | S_t, \theta)} \right]
\end{aligned} \tag{18}$$

$$\nabla_\theta \pi(a | s, \theta) = \pi(a | s, \theta) \nabla_\theta \log \pi(a | s, \theta) \tag{19}$$

Using the equations (18) and (19) and the update rule for neural networks, we get the following expression, which is called the REINFORCE update. So, for each timestamp in the reinforcement learning process, we compute these 2 variables using a for loop.

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (20)$$

$$\theta \leftarrow \theta + \alpha_t G_t \nabla_{\theta} \log \pi(A_t | S_t, \theta) \quad (21)$$

4.4 Neural Networks Introduction

Neural networks are non-linear models which are trained to mimic the human brain. There is an input layer followed by hidden layers and at last an output layer. They are used mainly for regression and classification tasks. A basic illustration would be

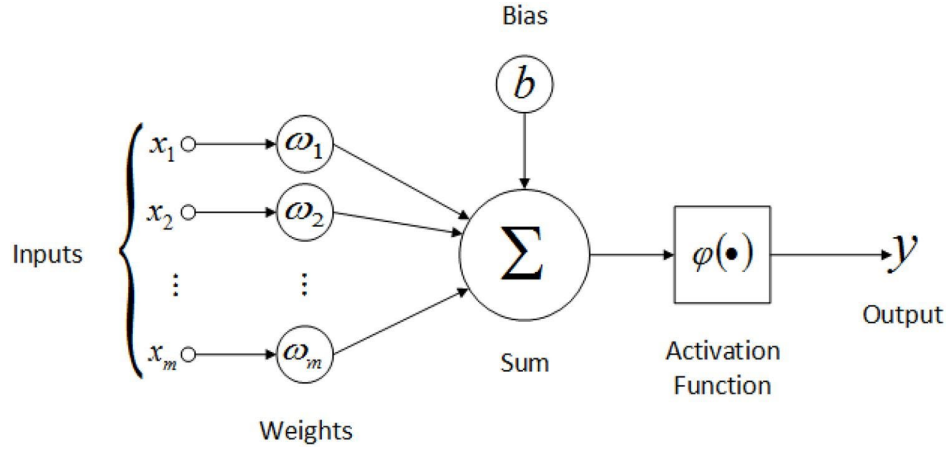


Figure 5: Simple neural network

Each element in the input vector is multiplied by a corresponding weight, and their sum is computed along with a bias term. This sum is then passed through an activation function. Each hidden layer has a predefined number of neurons and a specific activation function. The role of an activation function is to transform a linear input into a non-linear output. The choice of activation function depends on the nature of the problem being addressed. Some commonly used activation functions are

- **ReLU (Rectified Linear Unit):**

$$g(z) = \max(0, z)$$

- **Sigmoid:**

$$g(z) = \frac{1}{1 + e^{-z}}$$

- **Hyperbolic Tangent (Tanh):**

$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- **Softmax (for a vector $\mathbf{z} = [z_1, z_2, \dots, z_n]$):**

$$g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad \text{for } i = 1, 2, \dots, n$$

At the end of passing values through the hidden layers (which is called **forward propagation**), the output layer should provide the cost function and optimizing it should be the objective of our problem. **Backpropagation** is a part of the learning process in which the weights are fine-tuned based on the loss of error in the previous iteration. The gradient of the cost function with respect to the weights is calculated for this purpose.

5 Strategy Implementation

5.1 Technical Analysis Trading

We input the state matrix into our trading strategies to see which gives higher returns. The process begins at timestamp 200 to ensure that the 200-day moving average is included in every output. The algorithm runs for a total of 1000 days, after which the portfolio value is evaluated. **The first three strategies, i.e, moving averages, Bollinger bands and MACD do not involve any Reinforcement learning involved. The policy improvement method should be implemented on those strategies to make them learn by reinforcement.** We plan to conduct experiments with these 4 strategies by varying the number of days and comparing the returns in each case. We shall do this process sector-wise and tabulate the results.

```
[100000, [[0, 0, 0, 0, 0, 0], [np.float64(812.8544921875), np.float64(353.041259765625), np.float64(918.
Reward: 100000.0
[ np.float64(68824.5556640625), [[0, 0, 0, 0, 0, np.float64(2.0)], [np.float64(924.0801391601562), np.float64(
Reward: 100414.4033203125
[ np.float64(68824.5556640625), [[0, 0, 0, 0, 0, np.float64(2.0)], [np.float64(869.2783203125), np.float64(
Reward: 106580.7509765625
[ np.float64(43522.727783203125), [[np.float64(45.0), np.float64(24.0), 0, 0, 0, 0], [np.float64(997.8641
Reward: 102631.96984863281
[ np.float64(42519.57702636719), [[np.float64(25.0), np.float64(24.0), 0, np.float64(21.0), 0, np.float64(
Reward: 99765.43865966797
[ np.float64(33747.65377807617), [[np.float64(25.0), 0, 0, np.float64(21.0), np.float64(52.0), np.float64(
Reward: 101402.95883178711
[ np.float64(39925.649017333984), [[0, 0, np.float64(41.0), np.float64(12.0), np.float64(52.0), np.float64(
Reward: 110610.11849975586
[ np.float64(9679.280822753906), [[np.float64(11.0), np.float64(21.0), np.float64(41.0), np.float64(21.0)
Reward: 129376.03509521484
[ np.float64(9679.280822753906), [[np.float64(11.0), np.float64(21.0), np.float64(41.0), np.float64(21.0)
Reward: 161083.6612854004
[ np.float64(9679.280822753906), [[np.float64(11.0), np.float64(21.0), np.float64(41.0), np.float64(21.0)
Reward: 192646.34014892578
[ np.float64(126356.6782836914), [[0, np.float64(21.0), 0, np.float64(21.0), 0, np.float64(0.0)], [np.float64(
Reward: 174549.57751464844
```

Figure 6: Output for the Moving average trading strategy

```
[100000, [[0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [np.float64(812.8545532226562), np.float64(353.041259765625),
Reward: 100000
[ np.float64(20118.721435546875), [[0, 0, np.float64(91.0), 0, 0, 0], [np.float64(924.0801391601562), np.float64(
Reward: 104397.16284179688
[ np.float64(1456.3615417480469), [[np.float64(0.0), np.float64(7.0), np.float64(60.0), np.float64(0.0), np.float64(
Reward: 77499.77471923828
[ np.float64(2781.6426696777344), [[0, np.float64(0.0), 0, np.float64(0.0), np.float64(194.0), np.float64(0.0)],
Reward: 71576.85604858398
[ np.float64(33571.38983154297), [[np.float64(0.0), 0, np.float64(47.0), 0, 0, np.float64(1.0)], [np.float64(10
Reward: 76557.01336669922
[ np.float64(1262.4294738769531), [[np.float64(0.0), np.float64(109.0), np.float64(56.0), np.float64(0.0), 0, np
Reward: 60693.906341552734
[ np.float64(64128.6455078125), [[0, 0, 0, 0, 0, np.float64(0.0)], [np.float64(977.299072265625), np.float64(35
Reward: 64128.6455078125
[ np.float64(29009.390106201172), [[np.float64(8.0), np.float64(83.0), np.float64(0.0), np.float64(0.0), 0, np.f
Reward: 69186.04415893555
[ np.float64(3611.3099060058594), [[np.float64(50.0), 0, 0, np.float64(6.0), 0, np.float64(0.0)], [np.float64(1
Reward: 82433.16903686523
[ np.float64(3782.9043579101562), [[0, 0, 0, 0, np.float64(83.0), np.float64(0.0)], [np.float64(1638.65002441406
Reward: 91314.70233154297
[ np.float64(3345.2996215820312), [[np.float64(10.0), 0, np.float64(29.0), np.float64(19.0), np.float64(1.0), np
Reward: 84146.45007324219
```

Figure 7: Output for the Bollinger band trading strategy

These output images depict the state of the portfolio for every 100th timestamp, with the 1000th timestamp being the final portfolio.

```
[100000, [[0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [np.float64(812.8545532226562), np.float64(
Reward: 100000
[ np.float64(111320.34436035156), [[0, 0, 0, 0, 0, 0], [ np.float64(924.0801391601562), np.float64(642.949401855468
Reward: 111320.34436035156
[ np.float64(109414.81872558594), [[0, np.float64(9.0), 0, 0, 0, 0], [ np.float64(869.2782592773438), np.float64(46
Reward: 113617.7272644043
[ np.float64(33248.298248291016), [[0, 0, 0, 0, np.float64(220.0), np.float64(0.0)], [ np.float64(997.8642578125),
Reward: 111263.48867797852
[ np.float64(85338.64712524414), [[ np.float64(27.0), 0, 0, 0, 0, np.float64(0.0)], [ np.float64(1032.417236328125),
Reward: 113213.91250610352
[ np.float64(79149.14334106445), [[0, np.float64(77.0), 0, 0, 0, np.float64(0.0)], [ np.float64(1000.2244262695312)
Reward: 105083.99252319336
[ np.float64(58463.592681884766), [[0, 0, np.float64(30.0), np.float64(1.0), np.float64(46.0), np.float64(0.0)], [
Reward: 106796.80722045898
[ np.float64(1134.1250610351562), [[0, np.float64(52.0), np.float64(19.0), np.float64(22.0), np.float64(63.0), np.
Reward: 105060.26000976562
[ np.float64(126720.81927490234), [[ np.float64(0.0), 0, 0, 0, 0, np.float64(0.0)], [ np.float64(1397.30224609375),
Reward: 126720.81927490234
[ np.float64(140361.94165039062), [[0, 0, 0, 0, 0, np.float64(0.0)], [ np.float64(1638.6500244140625), np.float64(4
Reward: 140361.94165039062
[ np.float64(8918.017456054688), [[ np.float64(49.0), np.float64(68.0), np.float64(1.0), np.float64(0.0), 0, np.flo
Reward: 121944.76428222656
```

Figure 8: Output for the M.A.C.D trading strategy

5.2 Greedy Policy

The greedy policy function is designed to optimize asset allocation decisions at each time step using historical financial data. Starting with a fixed initial capital of 100,000, it iteratively determines the most advantageous way to distribute investments across seven different assets. A supporting function estimates the maximum number of shares that can be purchased for each asset, given the current available funds. These updates are systematically recorded and displayed, along with a calculated reward to assess performance. By applying dynamic programming, this method offers a structured and efficient strategy for making automated investment decisions in a multi-asset environment.

```
[100000, array([ 0. , 0. , 0. , 0. , 0. , 0. , 0. ,
0. , 0. , 0. , 0. , 0. ,
499.25976562, 427.18737793, 571.91394043, 77.92504883,
327.61538696, 16242.89257812]))
Reward: 100000.0
[100000, array([ 0. , 0. , 0. , 0. , 0. , 0. , 0. ,
0. , 0. , 0. , 0. , 0. ,
784.46813965, 488.2661438 , 699.60131836, 232.31349182,
364.37670898, 15838.97167969]))
Reward: 100000.0
[100000, array([ 0. , 0. , 0. , 0. , 0. , 0. , 0. ,
0. , 0. , 0. , 0. , 0. ,
918.88201904, 537.71685791, 812.85455322, 353.04125977,
446.64285278, 13956.61523438]))
Reward: 100000.0
[100000, array([ 0. , 0. , 0. , 0. , 0. , 0. , 0. ,
0. , 0. , 0. , 0. , 0. ,
926.13671875, 646.12426758, 924.08013916, 642.94940186,
682.86639404, 15771.41894531]))
Reward: 100000.0
[ np.float64(2133.8692016601562), array([ 21. , 30. , 27. , 26. ,
37. , 0. , 869.27825928, 466.98983765,
614.26654053, 742.74786377, 443.43817139, 18850.00390625]))
Reward: 86702.2611694336
[ np.float64(1373.1341552734375), array([ 21. , 30. , 27. , 26. ,
39. , 0. , 997.86425781, 591.88989258,
660.85638428, 900.855896 , 354.61450195, 16752.72265625]))
Reward: 95180.32159423828
[ np.float64(1373.1341552734375), array([ 21. , 30. , 27. , 26. ,
39. , 0. , 1032.41723633, 550.94714355,
536.53076172, 867.2711792 , 366.15979004, 17768.67773438]))
Reward: 90897.92346191406
[ np.float64(1373.1341552734375), array([ 21. , 30. , 27. , 26. ,
39. , 0. , 1000.22442627, 336.81622314,
405.68765259, 981.86785889, 423.66287231, 19911.35546875]))
Reward: 85487.31677246094
[ np.float64(1373.1341552734375), array([2.10000000e+01, 3.00000000e+01, 2.70000000e+01, 2.60000000e+01,
3.90000000e+01, 0.00000000e+00, 9.77299072e+02, 3.55978699e+02,
7.09254883e+02, 1.01388263e+03, 5.66123596e+02, 2.26038164e+04]))
Reward: 100165.42608642578
[ np.float64(1373.1341552734375), array([2.10000000e+01, 3.00000000e+01, 2.70000000e+01, 2.60000000e+01,
3.90000000e+01, 0.00000000e+00, 1.18227551e+03, 3.70101807e+02,
1.01468567e+03, 1.17377856e+03, 6.28233093e+02, 2.33603359e+04]))
Reward: 119719.82049560547
[ np.float64(1373.1341552734375), array([2.10000000e+01, 3.00000000e+01, 2.70000000e+01, 2.60000000e+01,
3.90000000e+01, 0.00000000e+00, 1.39730225e+03, 4.36186127e+02,
1.13138005e+03, 1.49279114e+03, 9.45042908e+02, 2.55263008e+04]))
Reward: 150016.40942382812
```

Figure 9: Output for the greedy algorithm trading strategy.

Another approach, which may or may not be more efficient, is known as the ϵ -greedy algorithm. In this method, we assign a greedy threshold, ϵ . The algorithm selects the optimal action with a probability of ϵ and opts for a random action with a probability of $1 - \epsilon$.

5.3 Monte Carlo policy gradient implementation

For this particular implementation, I have **limited the trading to just five stocks, that is, almost only five stocks can be bought or sold from each given company**. If we consider all trading possibilities with different numbers of stocks traded from each company, then the computational complexity of the algorithm would take a long processing time. In this Algorithm in each state we use the actions as the classes and so we use the softmax function to select a particular action. In the **REINFORCE** algorithm we use a neural network in which the **first two hidden layers are RELU followed by a softmax layer and finally a log-var layer**.

```
# Hyperparameters
num_episodes = 1001
gamma = 0.99
learning_rate = 0.001
initial_cash = 100000
num_stocks = 6

# Policy Network
class PolicyNetwork(tf.keras.Model):
    def __init__(self):
        super(PolicyNetwork, self).__init__()
        self.dense1 = tf.keras.layers.Dense(64, activation='relu')
        self.dense2 = tf.keras.layers.Dense(64, activation='relu')
        self.mean_layer = tf.keras.layers.Dense(num_stocks, activation='softmax')
        self.logvar_layer = tf.keras.layers.Dense(num_stocks)

    def call(self, state):
        x = self.dense1(state)
        x = self.dense2(x)
        mean = self.mean_layer(x)
        logvar = self.logvar_layer(x)
        return mean, logvar

policy = PolicyNetwork()
optimizer = tf.keras.optimizers.Adam(learning_rate)
```

Figure 10: Neural Network model in Tensorflow for REINFORCE.

In this optimization problem, the reward is defined as the logarithm of the ratio of returns between the current and the previous state. This formulation justifies the incorporation of the log-variance (log-var) layer within the neural network architecture, as it aligns with the reward structure.

6 Observation and Inference

Each of the four implemented strategies yields varying return percentages across different timestamps. The image below represents the outputs we got by using the REINFORCE strategy. Refer to figures 6,7,8,9 for the output of the other strategies.

```

[1.0000000e+05 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 0.0000000e+00 5.7191394e+02 7.7925056e+01 4.9925977e+02
4.2718741e+02 3.2761539e+02 1.6242890e+04]
Episode 0: Final Portfolio Value = 100000.0
[3.8847469e+04 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 4.1000000e+01 5.7000000e+01 1.5000000e+01 5.0000000e+00
7.0000000e+00 1.0000000e+00 6.9960132e+02 2.3231348e+02 7.8446808e+02
4.8826614e+02 3.6437674e+02 1.5838971e+04]
Episode 100: Final Portfolio Value = 113370.94916534424
[3.4074746e+03 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 2.7000000e+01 7.2000000e+01 1.2000000e+01 4.9000000e+01
1.4000000e+01 2.0000000e+00 8.1285461e+02 3.5304126e+02 9.1888196e+02
5.3771680e+02 4.4664282e+02 1.3956615e+04]
Episode 200: Final Portfolio Value = 122314.45650482178
[8.742593e+02 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 6.5000000e+01 4.5000000e+01 5.0000000e+00 4.1000000e+01
3.1000000e+01 1.0000000e+00 9.240001e+02 6.429494e+02 9.261367e+02
6.461242e+02 6.028664e+02 1.577142e+04]
Episode 300: Final Portfolio Value = 155454.2416305542
[2.7672049e+04 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 6.1000000e+01 2.8000000e+01 8.0000000e+00 1.1000000e+01
4.8000000e+01 1.0000000e+00 8.6927826e+02 4.6698981e+02 6.1426654e+02
7.4274786e+02 4.4343817e+02 1.8850004e+04]
Episode 400: Final Portfolio Value = 146993.1317062378
[1.1601466e+04 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 6.4000000e+01 1.9000000e+01 2.6000000e+01 8.0000000e+00
6.1000000e+01 1.0000000e+00 9.9786420e+02 5.9188989e+02 6.6085651e+02
9.0085583e+02 3.5461447e+02 1.6752723e+04]
Episode 500: Final Portfolio Value = 149484.00383758545
[6.3306626e+03 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 1.0900000e+02 5.0000000e+00 2.0000000e+01 3.0000000e+00
4.4000000e+01 0.0000000e+00 1.0324172e+03 5.5094720e+02 5.3653070e+02
8.6727118e+02 3.6615979e+02 1.7768678e+04]
Episode 600: Final Portfolio Value = 151062.33559417725
[4.2394004e+04 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 5.9000000e+01 0.0000000e+00 7.0000000e+00 3.3000000e+01
2.3000000e+01 0.0000000e+00 1.0002244e+03 3.3681625e+02 4.0568765e+02
9.8186792e+02 4.2366287e+02 1.9911352e+04]
Episode 700: Final Portfolio Value = 146392.9448776245
[4.2311199e+04 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 1.4000000e+01 1.7000000e+01 5.2000000e+01 3.2000000e+01
5.1000000e+01 0.0000000e+00 9.7729907e+02 3.5597870e+02 7.0925488e+02
1.0138826e+03 5.6612360e+02 2.2603818e+04]
Episode 800: Final Portfolio Value = 160242.8247909546
[3.20766133e+04 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 2.10000000e+01 1.60000000e+01
7.80000000e+01 1.30000000e+01 6.10000000e+01 0.00000000e+00
1.18227551e+03 3.70101807e+02 1.01468567e+03 1.17377869e+03
6.28233093e+02 2.33603359e+04]
Episode 900: Final Portfolio Value = 195552.85262298584
[3.5093547e+04 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00
0.0000000e+00 2.5000000e+01 4.6000000e+01 8.9000000e+01 2.1000000e+01
1.5000000e+01 0.0000000e+00 1.3973022e+03 4.3618613e+02 1.1313000e+03
1.4927911e+03 9.4504291e+02 2.5526301e+04]
Episode 1000: Final Portfolio Value = 236300.62523651123

```

Figure 11: Output for REINFORCE algorithm.

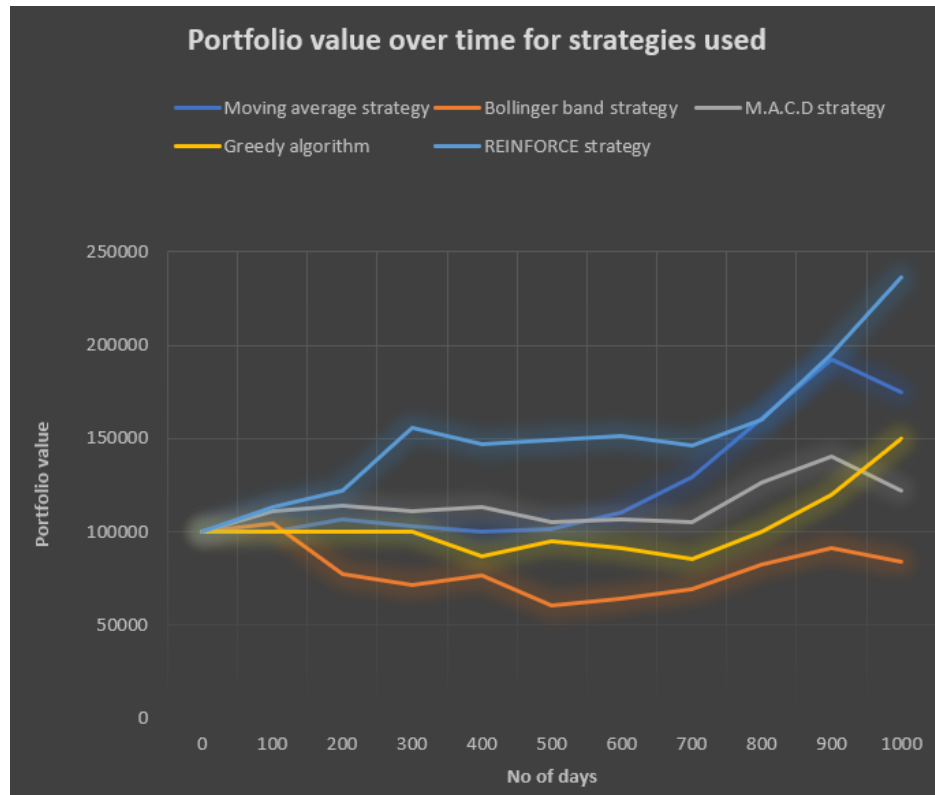


Figure 12: Portfolio value plot all trading strategies

Running the REINFORCE code many times, I got different portfolio values in episode(timestamp) number 1000. So we can interpret that it learns each time differently, but on average, the final portfolio value is higher than a final portfolio of the technical analysis strategies. Hence, we can conclude that this model is viable.

7 Source Code

- <https://colab.research.google.com/drive/1A3hy0F0z1X2NEqQxYC04SfY90jwCtVYB#scrollTo=1AAe28SMzKJv>
- https://colab.research.google.com/drive/1K0uxkBKtiiZl5Dr0cV9H0XK0S9Y1Wb_U#scrollTo=PrHlr5JoMCgQ

8 Future Work

I plan to extend this study by developing the Reinforcement learning model through hyperparameter tuning. Right now, the model trades with a limited number of stocks(to have a shorter computation time). It will require more training to help it adapt better to market conditions, like using more historical data of stocks and training with stocks of multiple sectors. I also want to work on reducing the computational complexity of the program so that the number of stocks can be significantly increased.

For the bot to remember each training it undergoes, we can use an inbuilt **cache(memory space allotment inside the Python program)**. So the model can get better over time by learning from different stocks through different periods.

9 References

- Technical Analysis: The Complete Resource for Financial Market Technicians by Charles D. Kirkpatrick ,Julie R. Dahlquist
- Reinforcement Learning :An Introduction, Second edition, Richard S. Sutton and Andrew G. Barto

- Deep Reinforcement Learning Strategies in Finance: Insights into Asset Holding, Trading Behavior, and Purchase Diversity, Regular Research Paper (CSCE-ICAI'24).Alireza Mohammadshafie,Akram Mirzaeinia,Haseebullah Jumakhan,Amir Mirzaeinia
- Portfolio Management using Deep Reinforcement Learning, Ashish Anil Pawar, Vishnureddy Prashant Muskawar, and Ritesh Tiku
- Nikolaos Pippas, Cagatay Turkay, and Elliot A. Ludvig. 2024. The Evolution of Reinforcement Learning in Quantitative Finance. ACM Comput. Surv. 1, 1, Article 1
- Grammarly was used as a reference tool to enhance the clarity, readability, and overall professionalism of this report.
- <https://stackoverflow.com/questions/65087771/reinforcement-learning-on-tensorflow-without-gym>